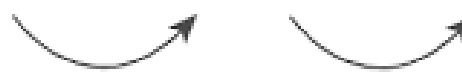


dict = { **k** : **v** , **k2** : **v2** , **k3** : **v3** }



ЛЕКЦІЯ №7 #2

ОПЕРАЦІЇ НАД СЛОВНИКАМИ

Аналіз словника.
Модифікація словника.
Обхід словників

ПЛАН

1. Перевірка приналежності ключа або значення до словника.
2. Вибір елементів чи колекцій словника.
3. Аналіз словника.
4. Модифікація словника.
5. Обхід словників.

1. ПЕРЕВІРКА ПРИНАЛЕЖНОСТІ КЛЮЧА АБО ЗНАЧЕННЯ ДО СЛОВНИКА

Таблиця 6.1. Перевірка приналежності ключа або значення до словника

Операція	Опис
<code>key in d</code>	Повертає True , якщо ключ <code>key</code> входить до <code>d</code>
<code>key not in d</code>	Повертає True , якщо ключ <code>key</code> не входить до <code>d</code>
<code>v in d.values()</code>	Повертає True , якщо значення <code>v</code> входить до <code>d</code>
<code>v not in d.values()</code>	Повертає True , якщо значення <code>v</code> не входить до <code>d</code>

1. ПЕРЕВІРКА ПРИНАЛЕЖНОСТІ КЛЮЧА АБО ЗНАЧЕННЯ ДО СЛОВНИКА

```
>>> d = {'first': 1, 'second': 2}
>>> 'first' in d
True
>>> 'third' in d
False
>>> 222 in d.values()
False
>>> 2 in d.values()
True
```

2. ВИБІР ЕЛЕМЕНТІВ ЧИ КОЛЕКЦІЙ СЛОВНИКА

Операція	Опис
<code>d.get(k)</code>	Повертає значення ключа <code>k</code> або None , якщо ключа <code>k</code> немає у словнику.
<code>d.get(k, v)</code>	Повертає значення ключа <code>k</code> або <code>v</code> , якщо ключа <code>k</code> немає у словнику.
<code>d.items()</code>	Повертає колекцію всіх пар ключ-значення в словнику <code>d</code> .
<code>d.keys()</code>	Повертає колекцію всіх ключів словника <code>d</code> .
<code>d.values()</code>	Повертає колекцію всіх значень в словнику <code>d</code> .

2. ВИБІР ЕЛЕМЕНТІВ ЧИ КОЛЕКЦІЙ СЛОВНИКА

```
>>> d = {'first': 1, 'second': 2}
>>> d.get('first')
1
>>> d.get("third", 3)
3
```

```
>>> d.items()
dict_items([('first', 1), ('second', 2)])
>>> d.values()
dict_values([1, 2])
```

3. АНАЛІЗ СЛОВНИКА

Операція	Опис
<code>len(d)</code>	Довжина <code>d</code> (кількість пар ключ-значення)
<code>min(d)</code>	Найменший ключ словника <code>d</code> (якщо ключі можна порівнювати)
<code>max(d)</code>	Найбільший ключ словника <code>d</code> (якщо ключі можна порівнювати)

```
>>> d = {'first': 1, 'second': 2}
>>> len(d)
2
>>> min(d)
'first'
```

Вище наведені операції можна проводити не лише з ключами, але і з значеннями словника. Для цього потрібно замість словника брати колекцію його значень.

```
min(d.values())
```

4. МОДИФІКАЦІЯ СЛОВНИКА.

Операція	Опис
<code>d.clear()</code>	Видаляє всі елементи зі словника <code>d</code>
<code>d.copy()</code>	Повертає копію словника <code>d</code>
<code>d.pop(k)</code>	Повертає значення ключа <code>k</code> і видаляє зі словника елемент з ключем <code>k</code> (якщо ключа <code>k</code> немає у словнику, то видає помилку)
<code>d.pop(k, v)</code>	Повертає значення ключа <code>k</code> і видаляє зі словника елемент з ключем <code>k</code> . Якщо ключа <code>k</code> немає у словнику, то повертає значення <code>v</code>
<code>d.popitem()</code>	Повертає і видаляє довільну пару ключ-значення зі словника <code>d</code> (якщо словник <code>d</code> порожній, то генерує помилку)
<code>d.update(d1)</code>	Додає в словник <code>d</code> пари ключ-значення з колекції <code>d1</code> , які відсутні в словнику <code>d</code> , а для кожного ключа, який вже присутній в словнику <code>d</code> , виконує заміну відповідним значенням з <code>d1</code> (колекція <code>d1</code> може бути як словником, так і будь-якою колекцією, що містить пари ключ-значення).

4. МОДИФІКАЦІЯ СЛОВНИКА.

```
>>> d = {'first': 1, 'second': 2}
>>> d.pop('first')
1
>>> d
{'second': 2}
>>> d = {'first': 1, 'second': 2}
>>> d1 = (('third', 3), ('forth', 4), ('first', 111))
>>> d.update(d1)
>>> d
{'forth': 4, 'third': 3, 'first': 111, 'second': 2}
>>> d.clear()
>>> d
{}
```

5. ОБХІД СЛОВНИКІВ

1. Обхід словника за ключами

```
for key in d:  
    process_iteration
```

Так, наприклад, результат виконання коду

```
>>> d = {'first': 1, 'second': 2, 'third': 3}  
>>> for key in d:  
    print(key)
```

стане виведення наступного тексту

```
second  
third  
first
```

5. ОБХІД СЛОВНИКІВ

2. Обхід словника за значеннями

```
for val in d.values():  
    process_iteration
```

```
>>> d = {'first': 1, 'second': 2, 'third': 3}  
>>> for val in d.values():  
    print(val)
```

```
2  
3  
1
```

5. ОБХІД СЛОВНИКІВ

3. Обхід словника за парами ключ-значення

```
for key, val in d.items():  
    process_iteration
```

```
>>> d = {'first': 1, 'second': 2, 'third': 3}  
>>> for key, val in d.items():  
        print(key, val)  
  
second 2  
third 3  
first 1
```

ЗАДАЧА. СЛОВА У РЯДКУ РОЗДІЛЯЮТЬСЯ ОДНИМ
АБО ДЕКІЛЬКОМА ПРОПУСКАМИ. ВИЗНАЧИТИ
КІЛЬКІСТЬ ВХОДЖЕНЬ КОЖНОГО СЛОВА ДО РЯДКА

1. Ввести рядок
2. Створити список слів
3. Обхід слів по списку
4. Створити порожній словник
5. При відсутності ключового слова додати пару: слово : кількість слів

КОД

```
S = input("введіть рядок: ")
words_list = S.split() # розбиваємо рядок на список слів
d = {}                 # порожній словник слів
for word in words_list: # пробігаємо по всіх словах
    if word not in d:    # якщо слово не входить до списку
                        # додаємо його до словника разом з
                        # його кількістю входжень
        d[word] = words_list.count(word)

print('слова та кількість входжень:', d)
```

ВИЗНАЧИТИ СЛОВО, ЯКЕ ВХОДИТЬ ДО ТЕКСТУ
НАЙБІЛЬШУ КІЛЬКІСТЬ РАЗІВ.

```
m = 0 # m - максимальна кількість входжень
max_word = '' # max_word - змінна що містить слово,
               # яке входить найбільшу кількість разів
for word, count in d.items(): # пробігаємо словник по
                              # всіх парах ключ-значення
    if m < count:
        m = count
        max_word = word

print('слово, яке входить найчастіше -', max_word)
```